Matrix Multiplication on a Weight-Stationary Systolic Architecture

A reference guide for VLSI & Embedded Systems Design

1. What is a Systolic Architecture?

A systolic array is a network of interconnected **Processing Elements (PEs)** that perform simple, repetitive operations like multiply-and-accumulate. Information flows directly between neighboring cells in a pipelined, rhythmic fashion—analogous to blood pumping through a heart.

- **The Goal:** General-purpose CPUs are inefficient for massive parallel tasks like deep learning. Systolic arrays decompose large mathematical operations into structured, parallel calculations.
- **Memory Efficiency:** They address the "Von Neumann bottleneck" by ensuring intermediate results are passed directly between PEs, drastically reducing main memory (SRAM/DRAM) read/write operations.

2. The Core Metric: Arithmetic Intensity

Deep learning workloads heavily rely on matrix-vector operations. Hardware acceleration is most effective when an application is *compute-bound* rather than *memory-bound*.

Arithmetic Intensity (AI) measures data reuse by calculating the ratio of arithmetic operations to data memory fetches.

For two $M \times M$ matrices, the intensity is:

$$AI = (2M - 1) / 3$$

Because matrix multiplication has inherently high arithmetic intensity, it is the perfect candidate for specialized hardware that avoids constant memory polling.

3. Inside the Hardware: The Processing Element

The fundamental building block of the systolic array is the Processing Element (PE), which is responsible for the core Multiply-Accumulate (MAC) operation.

$$S_{out} = S_{in} + (W \times X)$$

A typical PE features three critical data paths:

- **Weight (W):** Stored locally in the PE register.
- **Activation/Input (X):** Flows horizontally from left to right.
- **Partial Sum (S):** Flows vertically from top to bottom, accumulating results.

4. Dataflow Strategy: The Weight-Stationary Approach

This strategy defines how data moves through the array, framing the logic for custom animations of a matrix multiplication:

- **Pre-loaded Weights:** The neural network parameters (weights) remain stationary inside the individual PEs.
- **Streaming Activations:** The input data streams horizontally across the array, cycle by cycle.
- **Accumulating Sums:** Partial sums flow downwards, gathering the MAC results at each clock step.
- **Pipelining:** To prevent stalls, modern architectures pre-fetch and load the next set of weights in the background while the current computation runs.

5. Real-World Implementation: Google TPU

The concepts of the weight-stationary systolic array are effectively realized in Google's Tensor Processing Unit (TPU v1) and its Matrix Multiplier Unit (MXU).

- **Massive Scale:** The TPU v1 implements a massive **256 × 256** 2D systolic array.
- **Performance:** It can compute an astonishing 65,536 (256^2) multiply-accumulate operations in a single clock cycle, vastly outperforming traditional CPUs on matrix math.

Advantages

- **High Throughput:** Massive parallelism dedicated to matrix operations.
- **Power Efficiency:** Eliminates energy-intensive reads/writes of intermediate values to main RAM.
- **Silicon Efficiency:** Minimal control logic; silicon area is maximized for actual computation (MACs).

Disadvantages

- **Lack of Flexibility:** Struggles with complex branching or non-matrix math.
- **Utilization Drops:** If matrix sizes don't perfectly align with hardware dimensions, many PEs sit idle.